OpenClaw 架构解析

本文系统性地解释了 OpenClaw 为何以及如何被设计为一个 AI Agent 系统。文章从第一性原理出发，逐步构建 Agent 设计模式，然后以 OpenClaw 作为具体案例进行分析。

第一部分：为什么需要 Agent

裸 LLM 的问题

大语言模型（LLM）在其原始形态下，本质上是一个**无状态的文本补全引擎**。你给它一个提示词（Prompt），它返回一个回复，然后就彻底忘记一切。每一次调用都是一次独立的预测 —— 没有记忆、没有行动能力、也无法访问外部世界。

这造成了用户期望（"帮我完成这个任务"）与裸 LLM 实际能力（"这里是关于如何完成该任务的一段文字"）之间的根本性鸿沟。

试想当你向 LLM 提出以下请求时会发生什么：*"检查最新的日志文件中是否有错误，并修复你发现的 Bug。"*

裸 LLM 只能**告诉你**它会怎么做。它无法读取日志、无法打开代码、无法编辑文件、也无法验证修复是否正确。它没有手、没有眼睛、也没有记忆。

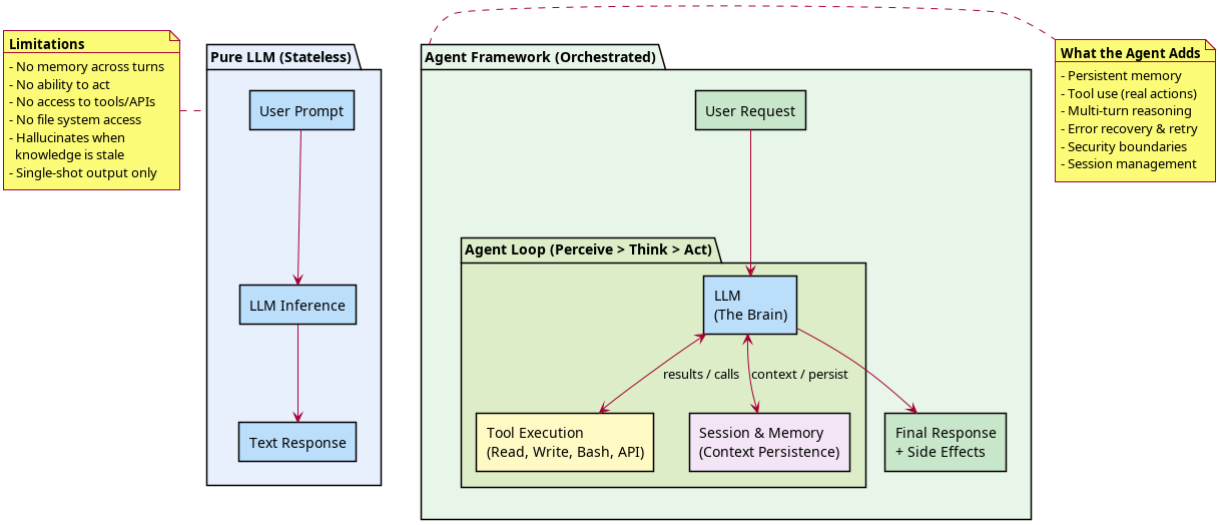
Agent 带来了什么

Agent 是包裹在 LLM 外层的运行时脚手架，赋予 LLM 在一个循环中**感知、推理和行动**的能力：

能力	裸 LLM	LLM + Agent
读取文件 / 执行命令	否	是（通过工具）
记住之前的对话	否	是（会话持久化）
调用外部 API	否	是（工具集成）
多步骤推理	仅单次推理	迭代循环
错误恢复	无	重试、故障转移、压缩
安全执行	无	沙箱、策略、白名单

Agent 充当一个**中间运行时循环**：它接收用户请求，将其连同可用的工具定义一起注入 LLM，让 LLM 推理下一步操作，截获 LLM 的工具调用决策，在真实世界中执行这些决策，并将结果反馈回去。这个循环不断重复，直到任务完成。

纯 LLM vs. LLM + Agent 框架



第二部分：大语言模型的特征

要设计一个有效的 Agent，你必须首先理解 LLM（Agent 内部的"大脑"）真正擅长什么，以及它在哪些方面存在根本性缺陷。

LLM 的优势

1. **自然语言理解：** LLM 擅长解析模糊的、依赖上下文的人类语言，并从中提炼出可执行的意图。
2. **推理与规划：** 在给定充分上下文的情况下，现代 LLM 能够将复杂任务分解为一系列逻辑步骤，并决定在每个阶段调用哪个工具。
3. **代码生成与转换：** LLM 在海量源代码上进行过训练，能够跨多种编程语言生成、编辑和调试程序。
4. **知识检索：** LLM 内化了截至其训练截止日期的广泛公共知识快照，使其能够提供领域专业知识而无需实时搜索。
5. **自适应沟通：** 它们能够根据对话上下文调整语气、正式程度、语言和细节层次。

LLM 的根本性局限

1. **无状态性：** 对 LLM 的每次调用都是独立的。它没有内置的先前交互记忆。如果没有外部会话/记忆系统，它无法从之前的对话轮次中学习或引用。
2. **无行动能力：** LLM 无法读取文件、执行代码、浏览网页或与任何系统交互。它只能*生成描述这些行动的文本*。
3. **幻觉（Hallucination）：** 当 LLM 缺乏知识（或问题超出其训练数据范围），它会自信地生成听起来合理但实际上不正确的信息，而不是承认不确定性。
4. **有限的上下文窗口：** LLM 每次调用只能处理有限数量的 Token。随着对话历史的增长，较早的上下文必须被总结（压缩）或修剪，从而产生丢失重要信息的风险。

5. **无内部验证能力：** LLM 无法将自己的输出与现实进行核对。它无法运行单元测试、验证文件是否正确写入、或确认命令是否成功执行。
6. **训练数据过时：** LLM 的知识冻结在其训练截止日期。它不知道该日期之后发布的事件、API 或库版本。

对 Agent 设计的启示

这些特征直接决定了任何 AI Agent 系统的架构：

- **无状态性**要求 Agent 提供**会话管理**和**记忆**层。
- **无行动能力**要求 Agent 提供**工具执行**层。
- **幻觉**要求 Agent 通过工具结果、文件内容和经过验证的数据提供**接地（Grounding）**。
- **有限的上下文窗口**要求 Agent 实现**压缩（Compaction）和上下文修剪**策略。
- **无内部验证能力**要求 Agent **观察工具结果**并将其反馈给 LLM 以进行自我纠正。
- **训练数据过时**要求 Agent 集成**网页搜索**和**实时 API** 工具。

第三部分：OpenClaw 如何被设计为 Agent（案例分析）

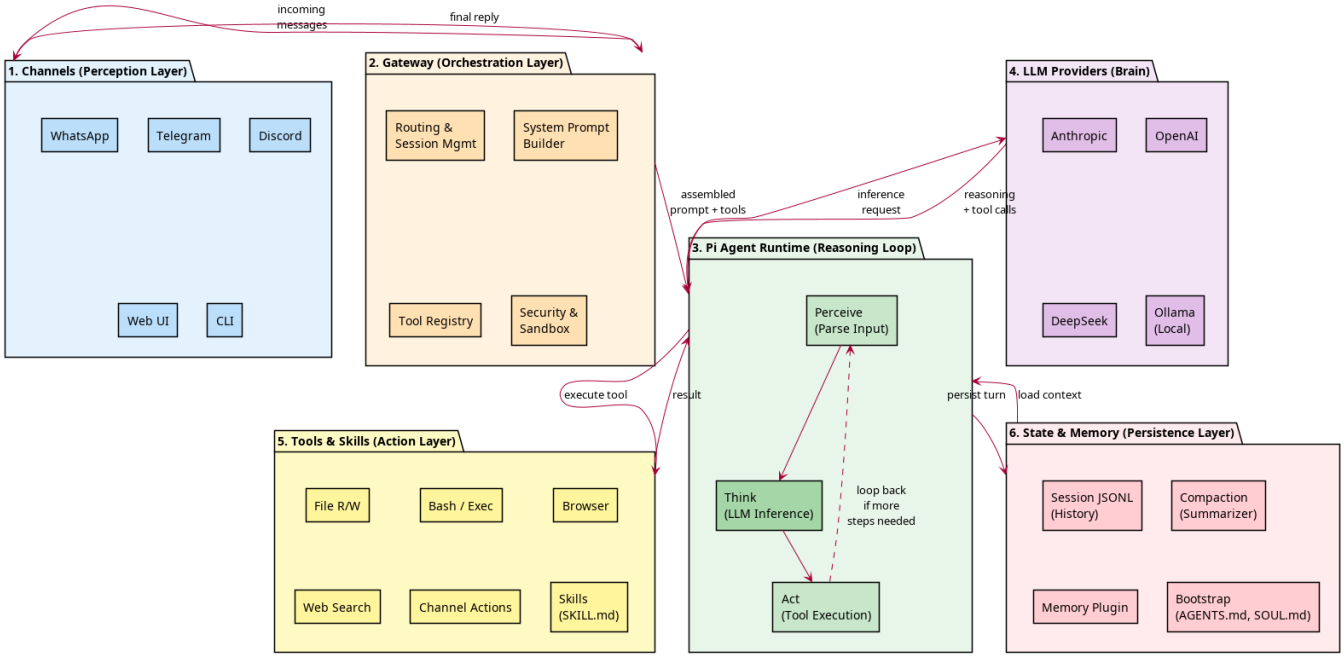
在理论基础到位之后，我们现在来考察 OpenClaw 如何在其架构中实现上述每一个 Agent 原则。

核心概念

OpenClaw 是一个本地优先（local-first）的个人 AI 助手编排系统。它设计为运行在用户自己的硬件上，充当一个中央"网关（Gateway）"，桥接各种大语言模型（LLM）、消息通道和客户端界面，使 AI 能够安全地执行本地任务。

OpenClaw 的 Agent 架构映射

下图展示了 OpenClaw 的组件如何映射到一个设计良好的 Agent 系统的六个层次：



Agent 原则	解决的 LLM 局限	OpenClaw 实现	Claude Code 实现（对比）
感知层	LLM 无法从多种来源接收输入	通道（WhatsApp、Telegram、Discord、Web UI、CLI）	终端 REPL、IDE 扩展（VS Code、JetBrains），以及自定义斜杠命令（`.claude/commands/`）
编排层	LLM 没有会话、路由或安全的概念	网关控制平面（路由、系统提示词构建、工具注册表、沙箱）	CLI 编排器，配合权限策略、工具访问控制，以及路径范围规则（`.claude/rules/`）用于编码规范
推理循环	LLM 仅进行单次推理	Pi Agent 运行时（感知-思考-行动循环，迭代式工具调用）	内置 Agent 循环（将 Prompt 和工具发送至 API、执行工具调用、反馈结果、重复）
大脑（可插拔）	不同的 LLM 有不同的优势	提供者抽象（Anthropic、OpenAI、DeepSeek、Ollama 等）	仅支持 Anthropic Claude 模型（Sonnet、Opus、Haiku）
行动层	LLM 无法在现实世界中行动	工具（文件读写、bash/exec、浏览器、网页搜索、通道操作）和技能	工具（Read、Write、Edit、Bash、Glob、Grep、WebFetch），技能（`.claude/skills/`），以及 MCP 服务器用于扩展

Agent 原则	解决的 LLM 局限	OpenClaw 实现	Claude Code 实现（对比）
持久化层	LLM 是无状态的	会话 JSONL 历史、压缩、记忆插件、引导文件	`CLAUDE.md` 项目上下文，规则和技能文件作为上下文注入，会话内记忆，`/compact` 用于上下文管理

各层如何解决特定的 LLM 局限

解决无状态性：会话与记忆

OpenClaw 将每一轮对话存储为结构化的 JSONL 文件。当新消息到达时，Pi Agent 运行时加载会话历史并将其注入 LLM 的上下文窗口。当上下文窗口接近限制时，OpenClaw 触发**自动压缩**来总结较早的对话轮次，在保持对话连贯性的同时不超出 Token 限制。

此外，引导文件（`AGENTS.md`、`SOUL.md`、`TOOLS.md`）在会话开始时被注入，以提供稳定的人格、操作指令和工具使用规范。

解决无行动能力：工具注入管道

OpenClaw 通过在系统提示词中注入一整套工具定义（JSON Schema）来赋予 LLM "双手"。LLM 推理应该使用哪个工具；Pi 运行时截获工具调用，在宿主机器上（或在 Docker 沙箱中）执行它，并将结果反馈到 LLM 的上下文中。

关键工具类别包括：

- **文件系统**（读、写、编辑）用于代码和文档操作
- **Bash/Exec** 用于运行任意命令
- **浏览器** 用于 Web 交互
- **网页搜索**（DuckDuckGo、Exa）用于实时信息检索
- **通道操作** 用于通过 WhatsApp、Telegram、Discord 等发送消息
- **技能**（从 `SKILL.md` 文件加载）用于专业化能力

解决幻觉：通过工具结果进行接地

Agent 循环不依赖 LLM 可能过时或不正确的内部知识，而是迫使 LLM **通过使用工具来验证其声明**。当 LLM 需要了解文件的内容时，它调用 `read` 工具。当它需要最新信息时，它调用 `web_search`。工具结果被注入回上下文中，提供基于事实的真相（Ground Truth），LLM 然后使用这些真相来构建其回应。

解决上下文限制：压缩与修剪

OpenClaw 实现了两种策略：

1. **自动压缩：** 当会话历史接近模型的上下文窗口限制时，Pi 触发压缩工作流，使用 LLM 本身将较早的对话轮次总结为紧凑摘要，在保留关键信息的同时减少 Token 数量。
2. **Cache-TTL 上下文修剪：** 一个自定义 Pi 扩展跟踪上下文项的"新鲜度"，并在过时条目消耗宝贵的上下文预算之前将其修剪。

解决安全性：沙箱与策略

由于 Agent 可以在用户的机器上执行任意代码，OpenClaw 实现了分层安全模型：

- **宿主优先（受信任模式）：** 对所有者的主会话进行直接执行。
- **沙箱模式：** 对不受信任/群组上下文进行 Docker 隔离执行。
- **工具策略过滤：** 白名单和黑名单控制每个通道、用户和会话类型可用的工具。

第四部分：Agent 开发者做什么

第一至第三部分解释了 Agent 架构的*是什么*和*为什么*。但谁来真正构建这些系统，他们的日常工作是什么样的？

Agent 开发的核心工作

构建像 OpenClaw 或 Claude Code 这样的 Agent 系统需要跨多个不同学科的工作：

工作领域	具体内容	OpenClaw 中的示例
Agent 循环设计	定义感知-思考-行动周期：何时循环、何时停止、如何处理多轮推理	Pi 运行时集成、轮次生命周期管理、流式事件分发
工具设计与实现	创建工具模式（JSON Schema）、实现工具后端、处理错误和超时	文件读写工具、bash 执行器、浏览器控制、通道特定操作（例如 `whatsapp_send`）
提示词工程	编写系统提示词、人格指令、安全护栏和工具使用规范	`AGENTS.md`、`SOUL.md`、`TOOLS.md`、`IDENTITY.md` —— 定义 Agent 角色的引导文件
上下文管理	设计压缩策略、记忆系统和上下文窗口优化	自动压缩工作流、Cache-TTL 修剪、`compaction-safeguard.ts`

工作领域	具体内容	OpenClaw 中的示例
安全与策略	定义沙箱边界、工具白名单/黑名单、信任模型和权限升级规则	Docker 沙箱集成、`tools.fs.workspaceOnly`、按通道工具过滤
通道与集成开发	为消息平台、API、设备硬件和外部服务构建适配器	80+ 扩展插件，覆盖 WhatsApp、Telegram、Discord、Ollama、浏览器、Hue、Sonos 等
评估与测试	构建基准测试、测试边界情况、衡量回复质量、检测回归	提示词评估、工具执行测试、端到端对话测试
可观测性与调试	日志记录、追踪、成本跟踪、Token 使用监控、调试非确定性 Agent 行为	会话事件日志、流式事件钩子、每会话 LLM 成本追踪

Agent 开发本身能由 AI 完成吗？

这是一个元问题：如果 Agent 已经强大到能编写代码、设计系统、推理架构 —— 它们能构建*自己*吗？

答案是**部分可以，且能力在持续增长**：

工作领域	AI/Agent 目前的能力	仍需人类参与的部分
工具实现	Agent（如 Claude Code）可以生成工具后端代码、编写 JSON Schema、实现错误处理	决定*构建哪些*工具、定义安全契约、判断边界情况的权衡
提示词工程	Agent 可以迭代提示词、测试变体、针对特定指标进行优化	定义 Agent 的*人格*、*价值观*和*信任模型* —— 这些是产品决策，而非代码
上下文管理	Agent 可以建议压缩策略并编写修剪启发式算法	为每种部署场景选择正确的策略需要对用户行为和成本约束的理解
代码生成	Agent 可以编写、编辑和调试代码库的大部分内容	架构决策、依赖选择和跨系统集成设计仍需要人类判断
测试	Agent 可以生成测试用例、运行基准测试、识别回归	定义*什么是"好的"* —— 评估标准本质上是主观的、由产品驱动的
安全	Agent 可以审计代码漏洞并建议沙箱配置	威胁建模、信任边界定义和监管合规需要人类承担责任

工作领域	AI/Agent 目前的能力	仍需人类参与的部分
通道集成	Agent 可以从文档生成 API 适配器代码	协商 API 访问权限、处理速率限制和管理认证凭证是运维任务
可观测性	Agent 可以对代码进行检测并设置日志记录	解释日志、诊断非确定性行为的根本原因、做出成本优化决策需要人类专业知识

新兴模式：人-Agent 协同开发

在实践中，Agent 开发越来越成为一种**人-Agent 协作**：

1. **人类设定方向：** 开发者定义 Agent 应该做什么、需要哪些工具、安全边界是什么、人格应该是怎样的。
2. **Agent 完成实现：** 开发者使用 AI 编码 Agent（如 Claude Code 本身）来编写工具后端、生成提示词模板、实现压缩逻辑、构建通道适配器。
3. **人类审查与迭代：** 开发者测试 Agent 的行为、观察边界情况、调整提示词和策略、进行架构修正。
4. **Agent 自我改进：** 高级系统可以使用自我评估循环，其中 Agent 测试自己的输出、识别弱点、并提出提示词或工具改进建议。

这不是遥远的未来 —— 这正是*此刻*用于构建 OpenClaw 的工作流。你正在阅读的这份文档本身，就是由一位人类开发者和一个 AI 编码 Agent 协作完成的。

第五部分：技术深入分析

第一至第四部分确立了*为什么*需要 Agent、LLM *带来了什么又缺少什么*、OpenClaw *如何*映射到 Agent 设计模式，以及*构建 Agent 的工作涉及哪些内容*。本部分深入探讨**实现细节**：代码库结构、SDK 集成和使模式具体化的配置层面细节。

代码库组织

第三部分描述的六个 Agent 层通过 OpenClaw monorepo 中的四个目录级别边界来实现：

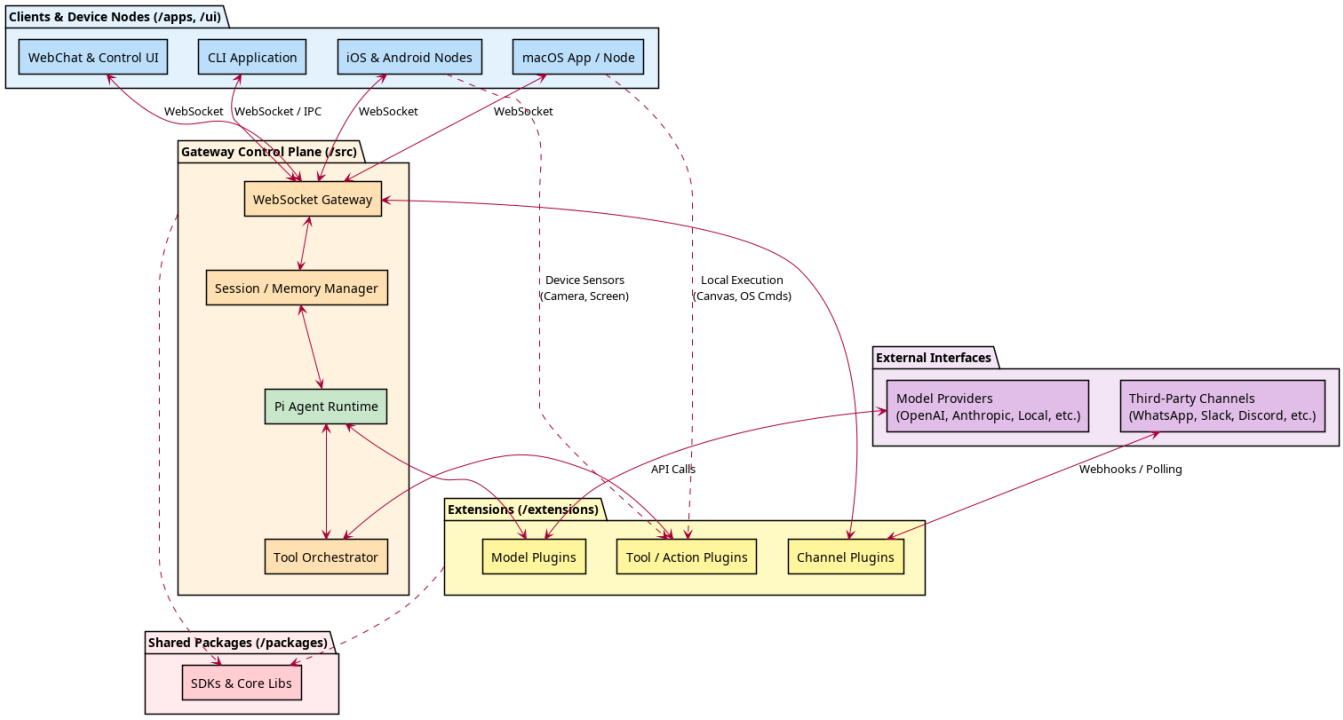
Agent 层（来自第三部分）	代码库位置	包含内容
编排层	`/src`	网关控制平面：WebSocket 服务器、路由引擎、会话管理、工具编排、系统提示词构建器

Agent 层（来自第三部分）	代码库位置	包含内容
感知层 + 大脑 + 行动层	`/extensions`	80+ 插件：通道适配器（WhatsApp、Telegram、Discord、Matrix 等）、LLM 提供者（Anthropic、OpenAI、DeepSeek、Groq、Ollama 等）和工具插件（浏览器、网页搜索、媒体理解）
感知层（客户端界面）	`/apps` 和 `/ui`	Web UI（基于 Vite 的 SPA，用于 WebChat 和仪表盘）、macOS/iOS/Android 原生节点（通过 WebSocket 暴露摄像头、麦克风、屏幕录制、Canvas UI），以及 CLI
共享基础设施	`/packages`	可复用 SDK：`clawdbot`、`memory-host-sdk`、`moltbot`

一个关键的架构决策：网关核心（`/src`）保持精简。领域特定能力（通道协议、模型提供者的特殊处理、专业化工具）被推送到 `/extensions` 中作为插件实现，使编排层与任何单一通道、模型或工具的具体细节解耦。

系统交互图

下图展示了消息在运行时如何在这些边界之间流动：



Pi Agent 运行时：实现细节

第三部分将**推理循环**引入为实现感知-思考-行动周期的层。在 OpenClaw 中，这个循环由 **Pi Agent 运行时**驱动，它是一个嵌入式 SDK 而非独立子进程。

OpenClaw 通过 `createAgentSession()` 直接将 pi-mono（具体是 `pi-coding-agent` 及其相关组件）导入网关的 Node.js 进程。这种嵌入式方法（相对于子进程 RPC）使 OpenClaw 能够对 Agent 生命周期的每个阶段进行程序化控制：

SDK 集成点

1. **推理循环（感知-思考-行动）：** Pi 引擎处理内部的逐轮循环。它将组装好的 Prompt 发送给 LLM 提供者，解析 `<think>` 推理块 vs `<final>` 答案，并在将最终文本返回给用户之前动态触发工具执行。这是第一部分中描述的迭代循环的具体实现。

2. **工具覆盖管道：** 如第三部分（"解决无行动能力"）所述，OpenClaw 将工具定义注入 LLM 的 Prompt。在实现层面，Pi 带有默认的基础工具（read、write、bash），但 OpenClaw **替换了所有这些工具**。它通过 `splitSdkTools()` 剥离 Pi 的默认实现，并注入自己的扩展工具集，包括浏览器控制、通道特定操作（例如 `whatsapp_send`）和伴侣应用传感器。所有工具都通过 `customTools` 流转，以便 OpenClaw 的策略过滤和沙箱集成统一生效。

3. **会话事件挂钩（实时流式传输）：** OpenClaw 订阅 Pi 的 `AgentSession` 事件（`message_start`、`message_update`、`tool_execution_start`、`tool_execution_end`、`turn_end`、`auto_compaction_start`）来将 AI 的内部思考过程转化为用户可见的输出。这使得 Web UI 中的实时流式更新和消息通道的分块回复成为可能。

4. **压缩实现：** 如第三部分（"解决上下文限制"）所述，OpenClaw 使用两种压缩策略。在实现层面，Pi 的 `SessionManager` 在 Token 上下文接近模型窗口限制时处理自动压缩。OpenClaw 在此基础上添加了两个自定义 Pi 扩展：一个是带有自适应 Token 预算和工具失败摘要的**压缩安全守卫**（`compaction-safeguard.ts`），另一个是**Cache-TTL 上下文修剪**（`context-pruning.ts`），它跟踪上下文项的新鲜度并修剪过时条目。

5. **动态 Prompt 组装：** 在会话开始时，系统提示词构建器（`buildAgentSystemPrompt()`）从 `~/.openclaw/workspace/` 读取本地维护的 `.md` 文件（`AGENTS.md`、`SOUL.md`、`TOOLS.md`、`IDENTITY.md`、`USER.md`，以及通过 `SKILL.md` 加载的技能），并将它们与工具定义、安全护栏和运行时元数据一起组装到系统提示词中。这就是第三部分中的"持久化层"如何将稳定的上下文注入每一个新的推理轮次。

工具职责分工：LLM vs Pi

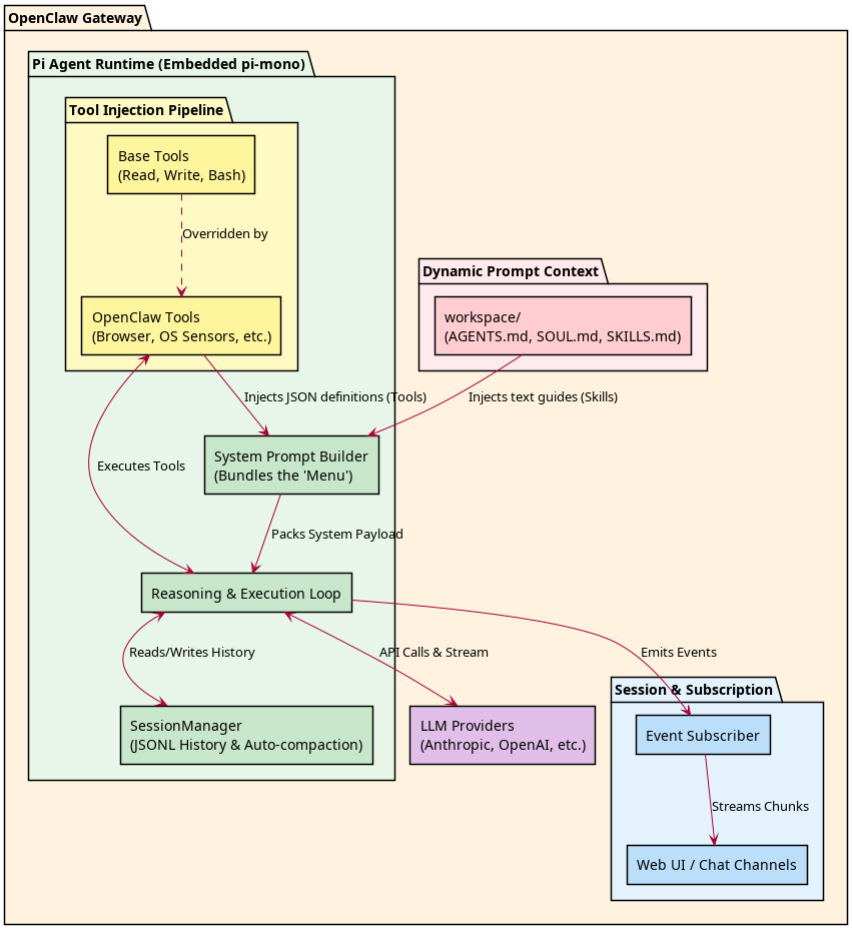
任何 Agent 系统中的一个关键设计边界是：**谁决定做什么** vs **谁来执行**。在 OpenClaw 中，这是清晰分离的：

1. **Pi 提供"菜单"：** 在每个 Prompt 之前，Pi 运行时将所有可用工具的 JSON Schema 定义打包并放入系统提示词。LLM 看到的是：*"这些是你可以调用的工具，这些是每个工具需要的确切参数。"*
2. **LLM 做决策：** 利用其推理能力（第二部分中的核心 LLM 优势），LLM 选择一个工具并在其响应流中发出结构化的工具调用请求。

3. **Pi 执行：** Pi 从 LLM 的流式输出中截获工具调用，暂停进一步的生成，运行实际的 TypeScript 后端代码，并将原始结果注入回 LLM 的上下文窗口。然后 LLM 继续使用新信息进行推理。这是第二部分（"对 Agent 设计的启示"）中描述的"观察工具结果并将其反馈回去"模式的具体实现。

这个循环在单个用户轮次内重复，直到 LLM 确定不再需要更多工具调用并发出最终回复。

Pi 运行时图



安全与执行模型：实现细节

第三部分在概念层面介绍了三级安全模型。这里我们讨论**信任模型的设计理由**和控制其运作的**配置层面细节**。

信任模型

OpenClaw 围绕**单一操作者信任模型（"个人助手"）设计。系统假设运行网关的人是机器的可信所有者。主要的安全关注点不是保护所有者免受 AI 的影响，而是防止外部提示词注入**（来自通过公共通道向机器人发消息的第三方）升级为宿主级别的操作。

执行层级的配置详情

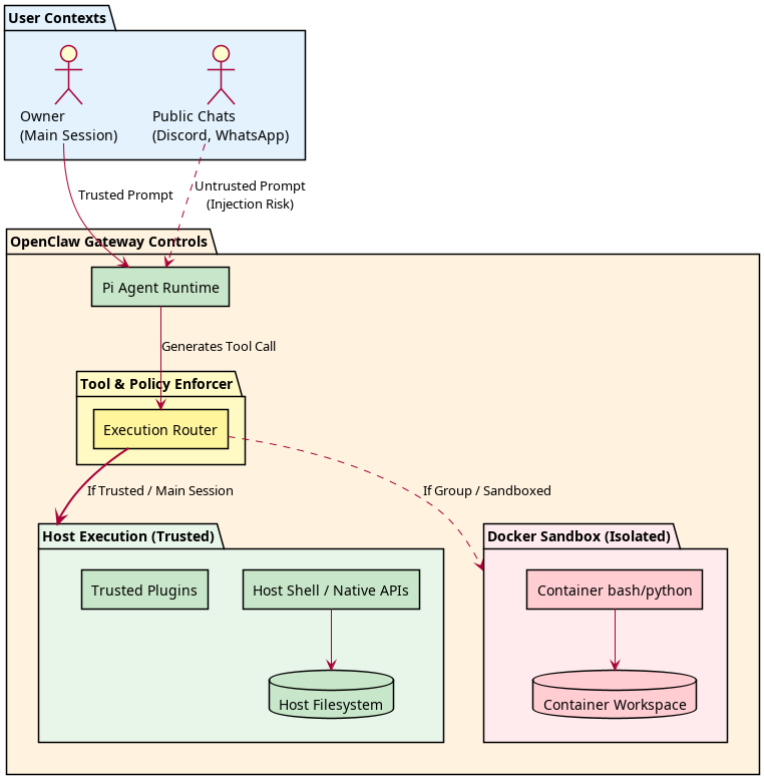
第三部分（"解决安全性"）中描述的三个层级由特定的配置键控制：

1. **宿主优先（受信任模式）：** 所有者主会话的默认模式。工具执行（`bash`、`python`、`fs`）以与网关进程相同的权限直接在宿主上运行。无需额外配置。
2. **沙箱模式：** 通过将 `agents.defaults.sandbox.mode` 设置为 `non-main`（沙箱化非所有者会话）或 `all`（沙箱化所有会话）来激活。激活后，本地执行工具被路由到一个没有宿主访问权限的 Docker 容器中，隔离来自公共 Telegram 群组或 Discord 服务器等通道的不受信任输入。
3. **受限工具范围：** 与沙箱模式正交。设置 `tools.fs.workspaceOnly: true` 强制所有文件系统工具仅在 `~/.openclaw/workspace` 内操作，无论信任级别如何。

插件信任边界

第三部分未涵盖的一个重要安全细微差别：从 `/extensions` 加载的自定义插件在**进程内**运行，作为可信计算基的一部分。它们在 Docker 沙箱之外执行，拥有与网关本身相同的宿主权限。这意味着安装恶意或被入侵的插件会绕过所有沙箱保护。因此，插件信任是一个操作者层面的决策，而非运行时强制执行的边界。

安全边界图



附录：有了 Claude Code，为什么还需要 OpenClaw

第三部分的对比表显示，OpenClaw 和 Claude Code 都实现了相同的六层 Agent 模式。如果 Claude Code 已经提供了一个具备工具、技能、规则和 MCP 扩展性的完整 Agent，为什么还要构建 OpenClaw?

答案是它们解决的是**根本不同的问题**：

维度	Claude Code	OpenClaw
核心定位	面向软件工程师的编码 Agent	面向所有人的通用个人助手
交互模式	基于会话：启动、使用、关闭	常驻网关：7×24 小时持续监听的持久化服务器
通道	仅终端和 IDE	WhatsApp、Telegram、Discord、Slack、Web UI、CLI、原生应用
受众	终端前的单个开发者	跨多个通道的多个用户同时使用
LLM 供应商	仅 Anthropic（供应商锁定）	任意提供者（Anthropic、OpenAI、DeepSeek、本地 Ollama 等）
设备集成	无	伴侣应用暴露摄像头、麦克风、屏幕录制、通知、Canvas UI
领域范围	代码：读取、编写、编辑、执行	生活：消息、媒体、智能家居、日历、编码，以及通过插件可达的一切

简而言之：

- **Claude Code** 是一个*编码工具* —— 你在有编程任务时打开它，完成后关闭它。
- **OpenClaw** 是一个*个人助手平台* —— 它在后台运行，桥接你的消息应用，控制你的设备，同时也具备编码能力。

关键架构差异在于：

1. **多通道存在：** OpenClaw 可以接收 WhatsApp 消息、对其进行推理、执行工具、并在 WhatsApp 上回复 —— 用户完全不需要接触终端。Claude Code 则要求用户在键盘前操作。
2. **常驻持久化：** OpenClaw 作为长期运行的服务器进程运行。它可以随时处理传入的消息，跨会话维护持续对话，并通过用户偏好的通道主动通知。
3. **提供者自由：** OpenClaw 的提供者抽象意味着你可以按会话、按通道甚至按消息在 Claude、GPT-4、DeepSeek 或本地 Ollama 模型之间切换 —— 根据成本、速度、隐私或能力需求进行优化。
4. **硬件桥接：** 通过 macOS、iOS 和 Android 上的伴侣应用，OpenClaw 将其"双手"延伸到任何基于终端的 Agent 都无法触及的物理设备能力。